

PLANIFICATOR

Architecture Technique & Schéma de Base de Données

Version 1.0 — Document de Référence

Projet	Planificator
Auteur	Firas HACHANI
Date	25/05/2026
Version	V1.0
Stack	Next.js 14 · TypeScript · PostgreSQL · Prisma · NextAuth v5
Statut	Architecture validée — prêt pour développement

Ce document présente l'architecture complète de l'application **Planificator**, une solution SaaS multi-entreprises de gestion de planning, d'équipes, d'employés, de clients et de chantiers. Il contient l'architecture technique recommandée, la structure des dossiers, le schéma de base de données Prisma complet, la matrice des rôles, les routes API, et le plan de développement.

1. Contexte & Besoins

Planificator est une application web de gestion de planning pensée dès le départ pour un déploiement multi-entreprises (SaaS). Chaque entreprise dispose de ses propres données isolées. L'objectif V1 est un usage interne, avec une commercialisation possible à terme.

Règles Métier Clés

- Une équipe contient généralement 5 ou 6 employés
- Chaque équipe a toujours un chef d'équipe obligatoire
- Un employé peut changer d'équipe
- Un employé ne peut pas être affecté à deux chantiers différents le même jour
- Une équipe ne peut pas être affectée à deux chantiers différents le même jour
- La planification se fait par journée complète (généralement 10h)
- L'admin crée les chantiers et attribue les équipes
- L'employé consulte uniquement son planning
- Le chef d'équipe consulte le planning de son équipe
- Le client consulte uniquement ses chantiers et le planning lié
- Archivage automatique d'un chantier terminé après 48h
- Pas d'inscription libre — invitation par email uniquement

Fonctionnalités V1 (20 points)

#	Fonctionnalité
01	Authentification email + mot de passe
02	Invitation utilisateur par email
03	Reset mot de passe
04	Gestion multi-entreprises
05	Gestion des rôles
06	Dashboard admin
07	Création des employés
08	Création des équipes
09	Chef d'équipe obligatoire
10	Création des clients
11	Création des chantiers
12	Ajout description, plan, photo et documents au chantier
13	Attribution d'une équipe à un chantier
14	Planning jour / semaine / mois

15	Planning par employé
16	Planning par équipe
17	Accès client limité à ses propres chantiers
18	Confirmation ou refus d'une affectation avec champ raison
19	Archivage automatique d'un chantier terminé après 48h
20	Fonction prolonger un chantier + historique des modifications

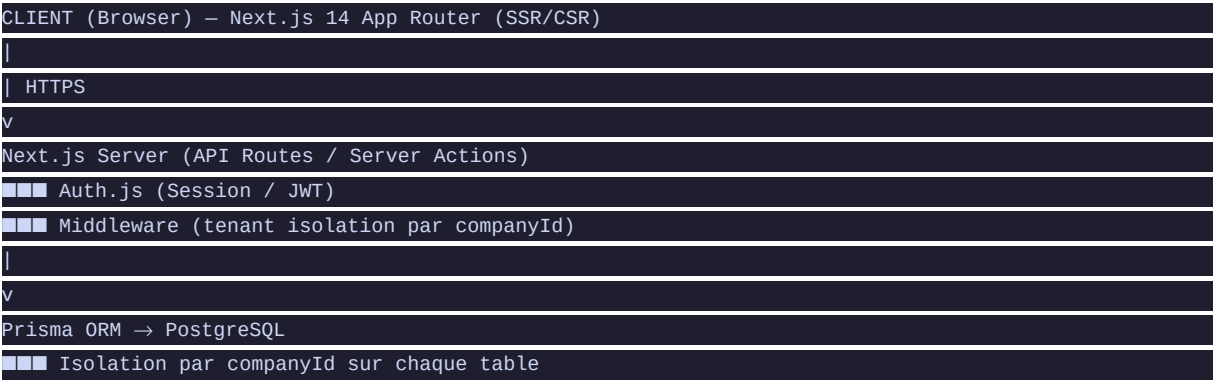
2. Architecture Technique

Stack Technologique

Couche	Technologie	Rôle
Framework	Next.js 14 (App Router)	SSR, routing, Server Actions
Langage	TypeScript	Typage statique
Base de données	PostgreSQL	BDD relationnelle
ORM	Prisma	Accès BDD typé
Authentification	NextAuth v5 (Auth.js)	Sessions, JWT, OAuth futur
UI	Tailwind CSS + shadcn/ui	Design system
Planning	react-big-calendar	Calendrier interactif
Email	Resend	Invitations, reset password
Upload	UploadThing	Fichiers, plans, photos
Validation	Zod	Schemas de validation
État serveur	Server Actions + React Query	Mutations et cache

Stratégie Multi-Tenant

Isolation par colonne **companyId** (row-level isolation). Simple, robuste, suffisant pour V1. Chaque requête Prisma filtre obligatoirement par companyId via un middleware Prisma. Le champ **slug** sur Company prépare la migration vers des sous-domaines (ex: acme.planificateur.fr).



3. Structure des Dossiers

```
planificator/
├── prisma/
│   ├── schema.prisma
│   └── seed.ts
├── src/
│   ├── app/
│   │   ├── (auth)/
│   │   │   ├── login/page.tsx
│   │   │   ├── reset-password/page.tsx
│   │   │   ├── invite/[token]/page.tsx
│   │   ├── (dashboard)/
│   │   │   ├── layout.tsx ← sidebar, navbar
│   │   │   ├── dashboard/page.tsx
│   │   │   ├── planning/
│   │   │   │   ├── page.tsx ← vue globale
│   │   │   │   ├── equipe/[id]/page.tsx
│   │   │   │   ├── employe/[id]/page.tsx
│   │   │   │   ├── equipes/
│   │   │   │   ├── employes/
│   │   │   │   ├── chantiers/
│   │   │   │   ├── clients/
│   │   │   │   ├── parametres/page.tsx
│   │   │   │   ├── (super-admin)/
│   │   │   │   ├── entreprises/
│   │   │   │   ├── (client-portal)/
│   │   │   │   └── mes-chantiers/page.tsx
│   │   │   └── api/
│   │   │       ├── auth/[...nextauth]/route.ts
│   │   │       ├── invitations/route.ts
│   │   │       ├── equipes/route.ts
│   │   │       ├── employes/route.ts
│   │   │       ├── chantiers/route.ts
│   │   │       ├── affectations/route.ts
│   │   │       └── planning/route.ts
│   │   └── components/
│   │       ├── ui/ ← shadcn auto-générés
│   │       ├── layout/
│   │       │   ├── Sidebar.tsx
│   │       │   ├── Navbar.tsx
│   │       └── planning/
│   │           ├── PlanningCalendar.tsx
│   │           ├── PlanningEquipe.tsx
│   │           ├── PlanningEmploye.tsx
│   │           └── chantiers/
│   └── shared/
├── lib/
│   ├── prisma.ts
│   ├── auth.ts
│   └── mail.ts
```

```
■ ■ ■ ■ ■ validations/
■ ■ ■ ■ ■ hooks/
■ ■ ■ ■ ■ types/
■ ■ ■ ■ ■ middleware.ts
■ ■ ■ ■ ■ .env.local
■ ■ ■ ■ ■ next.config.ts
■ ■ ■ ■ ■ package.json
```

4. Schéma Prisma Complet

4.1 Enums

```
enum Role {  
  SUPER_ADMIN // Accès total, gère les entreprises  
  ADMIN // Gère son entreprise  
  TEAM_LEADER // Chef d'équipe, confirme/refuse  
  EMPLOYEE // Consulte son planning  
  CLIENT // Consulte ses chantiers uniquement  
}  
  
enum WorksiteStatus {  
  PLANNED // Planifié, pas encore commencé  
  IN_PROGRESS // En cours  
  COMPLETED // Terminé (archivage auto 48h après)  
  ARCHIVED // Archivé définitivement  
  EXTENDED // Prolongé  
}  
  
enum AssignmentStatus {  
  PENDING // En attente de confirmation  
  CONFIRMED // Confirmé par le chef d'équipe  
  REFUSED // Refusé avec raison  
}  
  
enum DocumentType {  
  PLAN  
  PHOTO  
  DOCUMENT  
}  
  
enum HistoryAction {  
  CREATED | UPDATED | DELETED | ASSIGNED | UNASSIGNED  
  CONFIRMED | REFUSED | EXTENDED | ARCHIVED | INVITED | STATUS_CHANGED  
}
```

4.2 Company (Tenant)

```
model Company {  
  id String @id @default(cuid())  
  name String  
  slug String @unique // pour sous-domaine futur  
  logoUrl String?  
  address String?  
  phone String?  
  email String?  
  active Boolean @default(true)  
  createdAt DateTime @default(now())  
  updatedAt DateTime @updatedAt  
  
  // Relations  
  users User[]  
  teams Team[]  
  clients Client[]  
  worksites Worksite[]  
}
```

```

    invitations Invitation[]
  }
  @@map("companies")
}

```

4.3 User

```

model User {
  id String @id @default(cuid())
  email String @unique
  password String
  firstName String
  lastName String
  phone String?
  avatarUrl String?
  role Role @default(EMPLOYEE)
  active Boolean @default(true)
  companyId String? // null uniquement pour SUPER_ADMIN
  lastLoginAt DateTime?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  // Relations
  company Company? @relation(fields: [companyId], references: [id], onDelete: SetNull)
  teamMemberships TeamMember[]
  ledTeams Team[] @relation("TeamLeader")
  createdWorksites Worksite[] @relation("WorksiteCreatedBy")
  employeeAssignments EmployeeAssignment[]
  historyLogs HistoryLog[] @relation("HistoryActor")
  sentInvitations Invitation[] @relation("InvitedBy")
  clientProfile ClientProfile?
  passwordResets PasswordReset[]
}
@@map("users")

```

4.4 Team & TeamMember

```

model Team {
  id String @id @default(cuid())
  name String
  color String? // couleur hex pour le calendrier (#FF5733)
  companyId String
  leaderId String
  active Boolean @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  // Relations
  company Company @relation(fields: [companyId], references: [id], onDelete: Cascade)
  leader User @relation("TeamLeader", fields: [leaderId], references: [id])
  members TeamMember[]
  assignments Assignment[]
}
@@unique([name, companyId]) // pas de doublon dans la même entreprise
@@map("teams")

model TeamMember {

```

```
id String @id @default(cuid())
teamId String
userId String
joinedAt DateTime @default(now())
leftAt DateTime? // null = membre actif, sinon date de départ

// Relations
team Team @relation(fields: [teamId], references: [id], onDelete: Cascade)
user User @relation(fields: [userId], references: [id], onDelete: Cascade)

@@unique([teamId, userId])
@@map("team_members")
}
```

4.5 Client & ClientProfile

```
model Client {
  id String @id @default(cuid())
  name String
  email String?
  phone String?
  address String?
  notes String?
  companyId String
  active Boolean @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  // Relations
  company Company @relation(fields: [companyId], references: [id], onDelete: Cascade)
  worksites Worksite[]
  clientProfile ClientProfile?

  @@map("clients")
}

// Lien entre un Client et son compte User (accès portail)
model ClientProfile {
  id String @id @default(cuid())
  clientId String @unique
  userId String @unique

  // Relations
  client Client @relation(fields: [clientId], references: [id], onDelete: Cascade)
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@map("client_profiles")
}
```

4.6 Worksite (Chantier)

```
model Worksite {
  id String @id @default(cuid())
  name String
  description String?
  address String?
  latitude Float? // pour carte future
  longitude Float?
  status WorksiteStatus @default(PLANNED)
  startDate DateTime
  endDate DateTime
  dailyHours Float @default(10) // heures/jour
  clientId String
  companyId String
  createdById String
  archivedAt DateTime?
  completedAt DateTime?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  // Relations
  client Client @relation(fields: [clientId], references: [id])
  company Company @relation(fields: [companyId], references: [id], onDelete: Cascade)
```

```
createdBy User @relation("WorksiteCreatedBy", fields: [createdById], references: [id])
documents Document[]
assignments Assignment[]
extensions Extension[]
historyLogs HistoryLog[]
}
@@map("worksites")
}
```

4.7 Document

```
model Document {
  id String @id @default(cuid())
  worksiteId String
  name String
  url String
  size Int? // taille en octets
  mimeType String?
  type DocumentType @default(DOCUMENT)
  uploadedAt DateTime @default(now())

  // Relations
  worksite Worksite @relation(fields: [worksiteId], references: [id], onDelete: Cascade)

  @@map("documents")
}
```

4.8 Assignment & EmployeeAssignment (Planning)

```
model Assignment {
  id String @id @default(cuid())
  worksiteId String
  teamId String
  date DateTime @db.Date // journée complète
  status AssignmentStatus @default(PENDING)
  refusalReason String?
  notes String?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  // Relations
  worksite Worksite @relation(fields: [worksiteId], references: [id], onDelete: Cascade)
  team Team @relation(fields: [teamId], references: [id])
  employeeAssignments EmployeeAssignment[]

  @@unique([teamId, date]) // CONTRAINTE CLE : 1 équipe = 1 chantier/jour max
  @@map("assignments")
}

model EmployeeAssignment {
  id String @id @default(cuid())
  assignmentId String
  userId String
  date DateTime @db.Date // dupliqué pour la contrainte unique
  createdAt DateTime @default(now())

  // Relations
  assignment Assignment @relation(fields: [assignmentId], references: [id], onDelete: Cascade)
  user User @relation(fields: [userId], references: [id])

  @@unique([userId, date]) // CONTRAINTE CLE : 1 employé = 1 chantier/jour max
  @@map("employee_assignments")
}
```

4.9 Extension (Prolongation chantier)

```
model Extension {
  id String @id @default(cuid())
  worksiteId String
```

```

previousEndDate DateTime
newEndDate DateTime
reason String?
createdAt DateTime @default(now())

// Relations
worksite Worksite @relation(fields: [worksiteId], references: [id], onDelete: Cascade)

@@map("extensions")
}

```

4.10 HistoryLog (Audit Trail)

```

model HistoryLog {
  id String @id @default(cuid())
  worksiteId String?
  actorId String // qui a effectué l'action
  action HistoryAction
  entityType String? // "Worksite" | "Team" | "Assignment" | ...
  entityId String? // id de l'entité modifiée
  detail String? // description lisible
  metadata Json? // données avant/après (diff complet)
  createdAt DateTime @default(now())

  // Relations
  worksite Worksite? @relation(fields: [worksiteId], references: [id], onDelete: SetNull)
  actor User @relation("HistoryActor", fields: [actorId], references: [id])

  @@map("history_logs")
}

```

4.11 Invitation & PasswordReset

```

model Invitation {
  id String @id @default(cuid())
  email String
  role Role
  companyId String
  invitedById String
  token String @unique @default(cuid())
  expiresAt DateTime // recommandé : 24h
  usedAt DateTime?
  createdAt DateTime @default(now())

  // Relations
  company Company @relation(fields: [companyId], references: [id], onDelete: Cascade)
  invitedBy User @relation("InvitedBy", fields: [invitedById], references: [id])

  @@map("invitations")
}

model PasswordReset {
  id String @id @default(cuid())
  userId String
  token String @unique @default(cuid())
  expiresAt DateTime
  usedAt DateTime?
  createdAt DateTime @default(now())

  // Relations
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
}

```

```
@@map("password_resets")  
}
```

5. Diagramme des Relations

Company (tenant)
User[]
companyId Company
TeamMember[] Team (membre)
Team[] (leaderId "TeamLeader")
Worksite[] (createdById)
EmployeeAssignment[]
HistoryLog[] (actorId)
Invitation[] (invitedById)
ClientProfile? Client
PasswordReset[]
Team[]
companyId Company
leaderId User
TeamMember[] User
Assignment[]
worksiteId Worksite
teamId Team
EmployeeAssignment[]
assignmentId Assignment
userId User
Client[]
companyId Company
Worksite[]
ClientProfile? User
Worksite[]
companyId Company
clientId Client
createdById User
Document[]
Assignment[]
Extension[]
HistoryLog[]

Cardinalités

Relation	Type	Description
Company → User	1 → N	Une entreprise a plusieurs utilisateurs
Company → Team	1 → N	Une entreprise a plusieurs équipes
Company → Client	1 → N	Une entreprise a plusieurs clients
Company → Worksite	1 → N	Une entreprise a plusieurs chantiers

Team → User (leader)	N → 1	Chaque équipe a exactement un chef
Team ↔ User (members)	N ↔ N	Via TeamMember (avec date joinedAt/leftAt)
Client → Worksite	1 → N	Un client peut avoir plusieurs chantiers
Worksite → Assignment	1 → N	Un chantier peut avoir plusieurs affectations
Team → Assignment	1 → N	Unique par jour (contrainte @@unique)
User → EmployeeAssignment	1 → N	Unique par jour (contrainte @@unique)
Worksite → Document	1 → N	Plusieurs fichiers par chantier
Worksite → Extension	1 → N	Historique des prolongations
Client ↔ User	1 ↔ 1	Via ClientProfile (portail client)

6. Matrice des Rôles & Permissions

Fonctionnalité	Super Admin	Admin	Chef Equipe	Employé	Client
Gérer les entreprises	✓	✗	✗	✗	✗
Inviter des utilisateurs	✓	✓	✗	✗	✗
Gérer les équipes	✓	✓	✗	✗	✗
Gérer les chantiers	✓	✓	✗	✗	✗
Gérer les clients	✓	✓	✗	✗	✗
Voir planning global	✓	✓	✗	✗	✗
Voir planning de son équipe	✓	✓	✓	✗	✗
Voir son propre planning	✓	✓	✓	✓	✗
Confirmer/refuser affectation	✓	✓	✓	✗	✗
Prolonger un chantier	✓	✓	✗	✗	✗
Voir ses propres chantiers	✗	✗	✗	✗	✓
Voir planning lié ses chantiers	✗	✗	✗	✗	✓
Voir documents chantier	✓	✓	✓	✓	✓

7. Routes API Principales

Authentification

Méthode	Route	Description
POST	/api/auth/[...nextauth]	Login, logout, session
POST	/api/auth/reset-password	Demander un reset
PUT	/api/auth/reset-password/[token]	Valider le nouveau mot de passe

Invitations

Méthode	Route	Description
POST	/api/invitations	Inviter un utilisateur
GET	/api/invitations/[token]	Valider le token
POST	/api/invitations/[token]	Créer le compte depuis invitation

Entreprises (Super Admin)

Méthode	Route	Description
GET	/api/companies	Lister toutes les entreprises
POST	/api/companies	Créer une entreprise
GET	/api/companies/[id]	Détail entreprise
PUT	/api/companies/[id]	Modifier entreprise
DELETE	/api/companies/[id]	Désactiver entreprise

Équipes

Méthode	Route	Description
GET	/api/teams	Lister (filtrées par companyId)
POST	/api/teams	Créer une équipe
PUT	/api/teams/[id]	Modifier équipe / changer chef
DELETE	/api/teams/[id]	Archiver équipe
POST	/api/teams/[id]/members	Ajouter un membre
DELETE	/api/teams/[id]/members/[userId]	Retirer un membre

Employés

Méthode	Route	Description
GET	/api/employees	Lister les employés

POST	/api/employees	Créer un employé
PUT	/api/employees/[id]	Modifier employé
GET	/api/employees/[id]/planning	Planning d'un employé

Clients

Méthode	Route	Description
GET	/api/clients	Lister les clients
POST	/api/clients	Créer un client
PUT	/api/clients/[id]	Modifier client

Chantiers

Méthode	Route	Description
GET	/api/worksites	Lister les chantiers
POST	/api/worksites	Créer un chantier
GET	/api/worksites/[id]	Détail chantier
PUT	/api/worksites/[id]	Modifier chantier
POST	/api/worksites/[id]/extend	Prolonger un chantier
POST	/api/worksites/[id]/archive	Archiver manuellement
POST	/api/worksites/[id]/documents	Uploader un document

Affectations (Planning)

Méthode	Route	Description
GET	/api/assignments	?from=&to=&teamId=&employeeId=
POST	/api/assignments	Affecter une équipe à un chantier
PUT	/api/assignments/[id]/status	Confirmer ou refuser
DELETE	/api/assignments/[id]	Supprimer une affectation

8. Plan de Développement — 8 Étapes

Étape 1	Setup & Infrastructure 2-3h	• Init Next.js + TypeScript • Prisma + PostgreSQL + migrations • shadcn/ui + Tailwind configuration • Variables d'environnement
Étape 2	Authentification 3-4h	• NextAuth v5 (email/password) • Login page • Reset mot de passe • Invitation par email (Resend) • Middleware de protection des routes
Étape 3	Multi-tenant & Rôles 2-3h	• Middleware Prisma (tenant filter automatique) • Hook useCurrentUser • Composant RoleGate
Étape 4	Gestion Admin 4-5h	• Dashboard avec statistiques • CRUD Entreprises (Super Admin) • CRUD Employés • CRUD Équipes + assignation chef d'équipe
Étape 5	Chantiers 4-5h	• CRUD Chantiers • CRUD Clients • Upload documents/plans/photos • Archivage automatique (cron job)
Étape 6	Planning & Affectations 5-6h	• Calendrier react-big-calendar • Affectation équipe ↔ chantier • Contraintes : 1 chantier/équipe/jour • Confirmation / refus avec raison • Vues : globale, équipe, employé
Étape 7	Portail Client 2-3h	• Vue client (ses chantiers uniquement) • Planning lié à ses chantiers
Étape 8	Finitions V1 2-3h	• Historique des modifications (audit trail) • Fonction prolonger chantier • Archivage auto 48h post-complétion • Tests manuels + seed

9. Points de Vigilance

Securité

- Toujours valider companyId côté serveur — ne jamais le prendre depuis le client
- Le middleware Prisma doit filtrer CHAQUE requête par companyId
- Les tokens d'invitation expirent après 24h maximum
- Hasher les mots de passe avec bcrypt (minimum 12 rounds)
- Rate limiting sur les endpoints d'authentification

Contraintes Métier BDD

- @@unique([teamId, date]) sur Assignment est le filet de sécurité côté BDD — obligatoire
- @@unique([userId, date]) sur EmployeeAssignment — idem
- @db.Date sur date stocke uniquement la date sans heure — évite les conflits UTC
- Stocker toutes les dates en UTC, afficher en fuseau local côté client

Architecture Next.js 14

- NextAuth v5 (beta) a une API différente de v4 — suivre auth.js.dev
- Utiliser les Server Actions pour les mutations (moins de boilerplate que les routes API)
- Garder les composants clients ('use client') au minimum — privilégier Server Components

Scalabilité Future

- Le slug sur Company prépare le déploiement multi-sous-domaines
- Le schéma par colonne permet une migration vers des schémas séparés si besoin
- Prévoir une table Plan/Subscription pour la commercialisation
- Ne pas over-engineer la V1 — YAGNI

10. Commandes d'Installation

```
# 1. Créer le projet Next.js
npx create-next-app@latest planificator \
--typescript --tailwind --eslint \
--app --src-dir --import-alias "@/*"
cd planificator

# 2. ORM & Base de données
npm install prisma @prisma/client
npx prisma init

# 3. Authentification
npm install next-auth@beta @auth/prisma-adapter

# 4. Validation
npm install zod

# 5. Email (invitations + reset password)
npm install resend

# 6. Upload fichiers
npm install uploadthing @uploadthing/react

# 7. Planning calendrier
npm install react-big-calendar date-fns
npm install -D @types/react-big-calendar

# 8. shadcn/ui
npx shadcn@latest init
npx shadcn@latest add button input label card dialog table badge
npx shadcn@latest add select textarea avatar dropdown-menu
npx shadcn@latest add calendar popover sheet tabs

# 9. Utilitaires
npm install clsx tailwind-merge bcryptjs
npm install -D @types/bcryptjs

# 10. Première migration
npx prisma migrate dev --name init
npx prisma generate

# 11. Lancer l'application
npm run dev
```

Variables d'Environnement (.env.local)

```
# Base de données
DATABASE_URL="postgresql://user:password@localhost:5432/planificator"

# NextAuth
NEXTAUTH_URL="http://localhost:3000"
NEXTAUTH_SECRET="your-secret-key-here"

# Email (Resend)
RESEND_API_KEY="re_XXXXXXXXXXXX"
EMAIL_FROM="Planificator <noreply@planificator.fr>"

# Upload (UploadThing)
UPLOADTHING_SECRET="sk_live_XXXXXXXXXXXX"
UPLOADTHING_APP_ID="XXXXXXXXXXXX"
```

```
# App
NEXT_PUBLIC_APP_URL="http://localhost:3000"
INVITATION_EXPIRY_HOURS=24
```

Prochaines Étapes

Ce document valide l'architecture complète de Planificator V1. Une fois ce document approuvé, le développement peut commencer par l'Étape 1.

#	Action	Rôle
1	Valider ce document d'architecture	Admin
2	Créer le repository Git (GitHub/GitLab)	Admin
3	Configurer PostgreSQL local	Dev
4	Exécuter les commandes d'installation (Section 10)	Dev
5	Copier le schéma Prisma (Section 4) dans prisma/schema.prisma	Dev
6	Lancer <code>npx prisma migrate dev --name init</code>	Dev
7	Démarrer l'Étape 2 : Authentification	Dev

Document généré le 25/05/2026 — Planificator V1.0